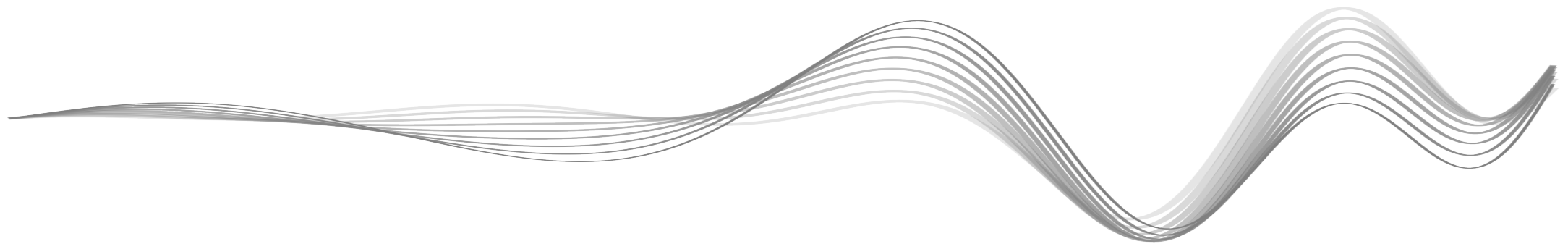




Recurrent Neural Networks





Basic RNNs

Feed-Forward Networks for Sequences

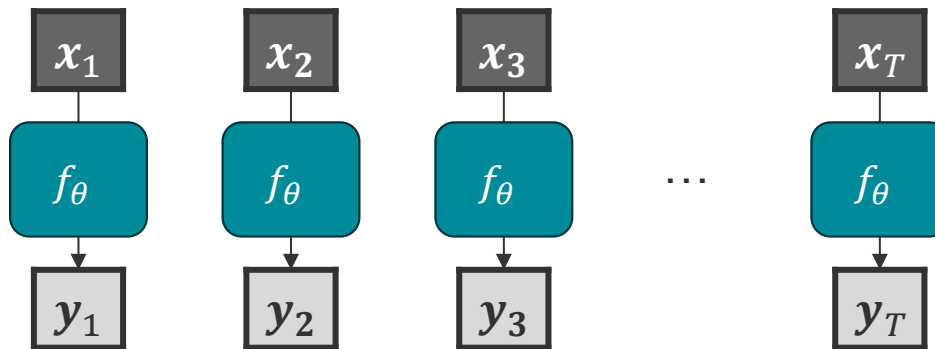
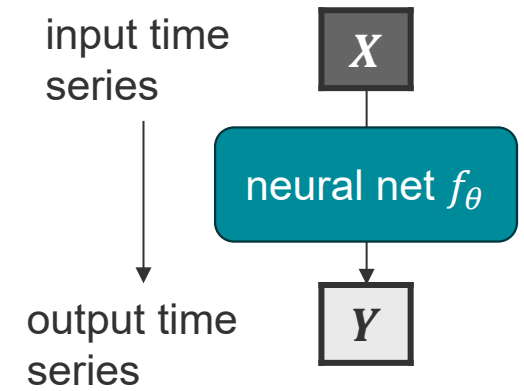


- **Learning sequential data**

- inputs: time series of d_x -dim. vectors $X = [x_1, x_2, \dots, x_T]^T \in \mathbb{R}^{(T \times d_x)}$
- outputs: scalars, vectors, time series

- How to use **feed-forward networks for sequence prediction tasks?**

- **Naïve approach:** consume inputs one-by-one



Shortcomings:

- No consideration of the inherent sequential character (independent predictions across time steps)
- No memory along time: prediction y_t only depends on x_t

Feed-Forward Networks for Sequences

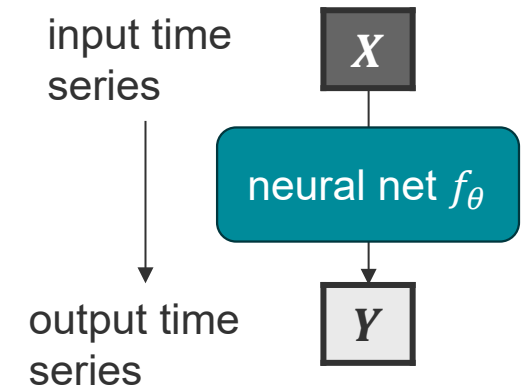
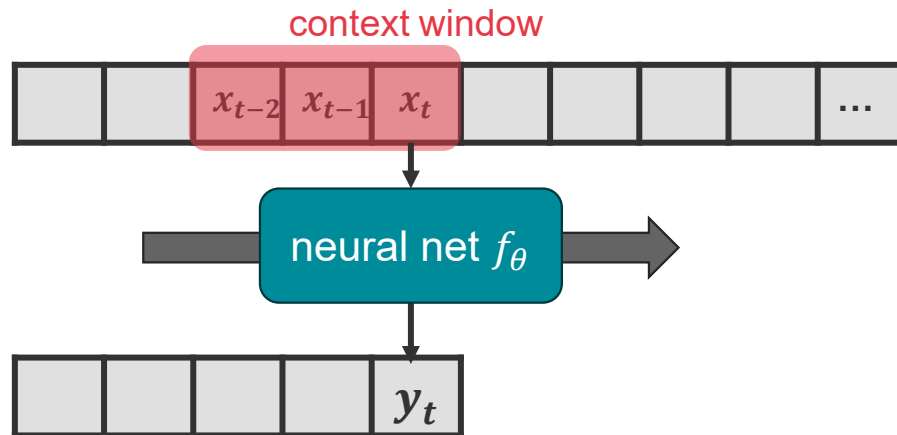


- **Learning sequential data**

- inputs: time series of d_x -dim. vectors $X = [x_{t_0}, x_{t_1}, \dots, x_T] \in \mathbb{R}^{(T \times d_x)}$
- outputs: scalars, vectors, time series

- How to use **feed-forward networks for sequence prediction tasks?**

- **Sliding window approach**



Properties:

- Limited memory (length of context wind.)
- Hard-coded context length (hyperparameter)
- Long context $\rightarrow$ many trainable parameters



- **Feedforward models do not allow for capturing long-term dependencies efficiently**
- Call for a **stateful model**
 - **Memory** that builds up along the input sequence
 - **Directionality**: information is added to the memory in positive time direction
- **Stateful models**, i.e. dynamic models that maintain memory between inputs. Information processing depends on previous inputs (i.e. entries of the input sequence at prior index).
- **Risk**: catastrophic forgetting: internal state does not keep track of relevant historic info

Stateful Models

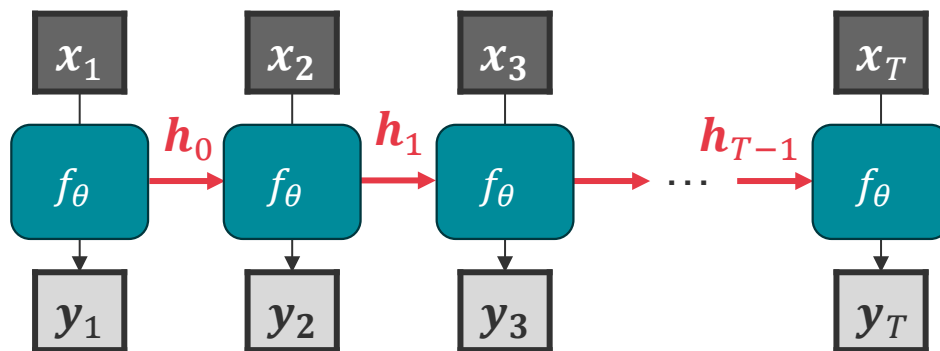
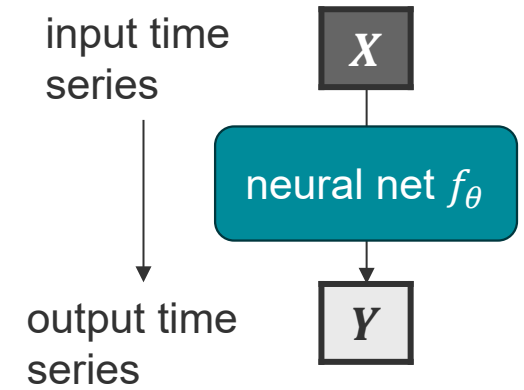


- **Learning sequential data**

- inputs: time series of d_x -dim. vectors $X = [x_1, x_2, \dots, x_T]^T \in \mathbb{R}^{(T \times d_x)}$
- outputs: scalars, vectors, time series

- **Neurons with recurrence**

- Link predictions at current time to previous times
- Keep track of the “**state**” h of the neural net at previous times



Properties:

- State creates a **memory** across the complete sequence
- Current predictions depend on previous inputs

$$\hat{y}_t = f_\theta(x_t, h_{t-1})$$

Recurrent Units

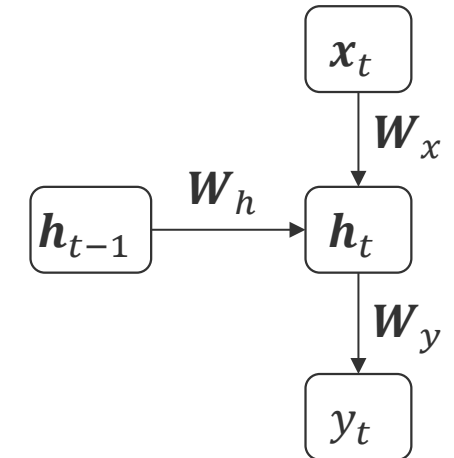


- Recap: **feed-forward neurons**

- layer index $(\cdot)^{(l)}$
- weight matrix $W^{(l)}$
- bias vector $b^{(l)}$
- activation function $g(\cdot)$
 g_1 typically chosen as tanh

$$x^{(l)} = g(W^{(l)}x^{(l-1)} + b^{(l)})$$

output activation function input trainable parameters



- Transition to recurrent units:** inputs and outputs are **sequential** $(\cdot)_t : y_t, x_t$

1. The state update compresses the current input and the previous state

$$h_t = g_1(W_x x_t + b + W_h h_{t-1})$$

2. The readout converts the state into the required prediction

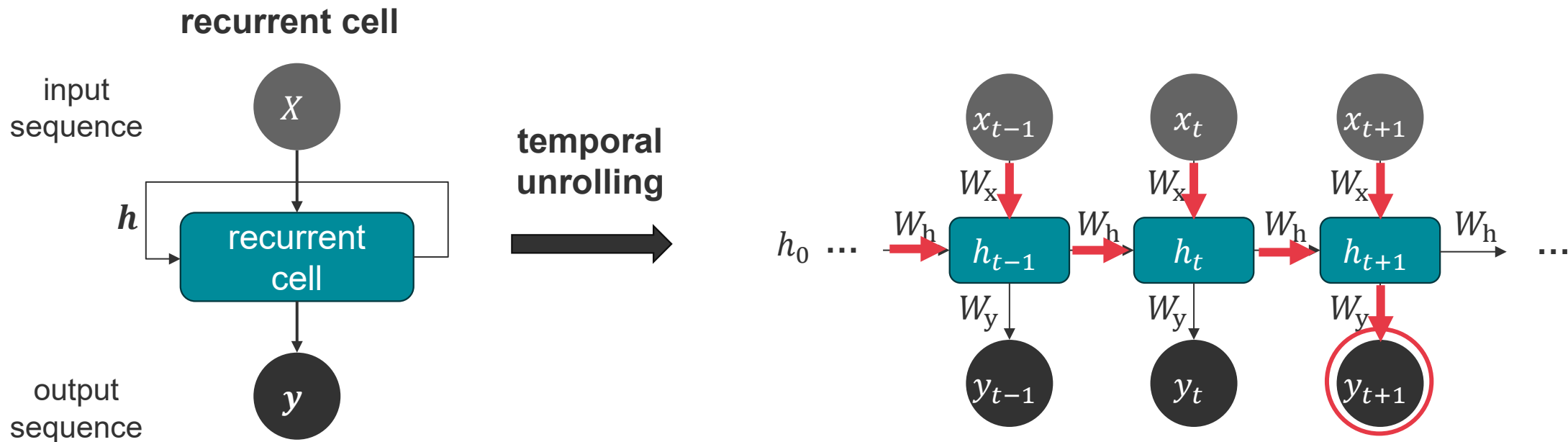
$$y_t = g_2(W_y h_t + b_y)$$

$W_x \in \mathbb{R}^{d_h \times d_x}$ input weights
 $W_h \in \mathbb{R}^{d_h \times d_h}$ state weights
 $W_y \in \mathbb{R}^{d_y \times d_h}$ output weights
 b bias; b_y output bias

Recurrent Units

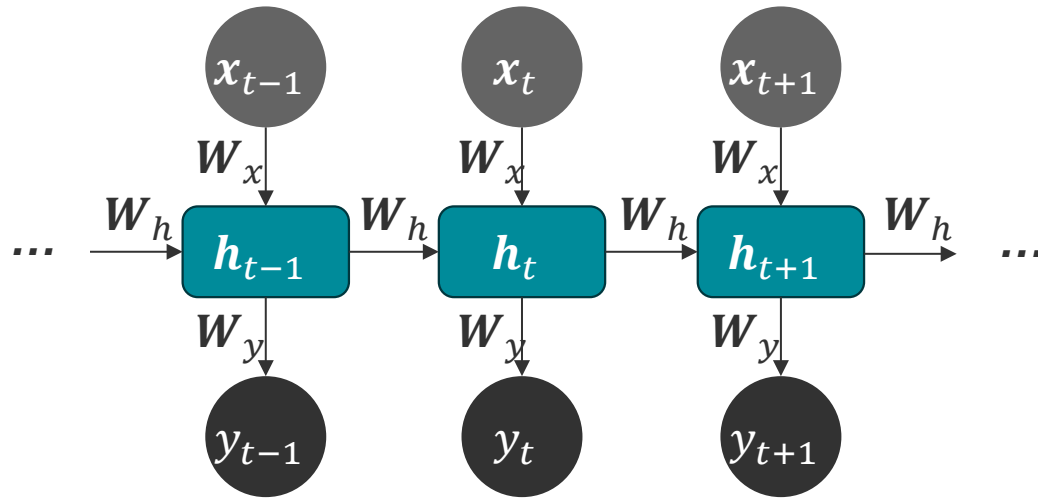


- Recurrent neurons („recurrent cell“, „recurrent unit“) maintain a **hidden state that depend on previous inputs, thereby encoding memory: The current output depends on previous inputs.**



Previous inputs affect current output
→ memory

Recurrent Units: Weight Sharing



- State update $\mathbf{h}_t = g_1(\mathbf{W}_x \mathbf{x}_t + \mathbf{b} + \mathbf{W}_h \mathbf{h}_{t-1})$

- Readout $\mathbf{y}_t = g_2(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$

Activation function $g_2(\cdot)$ depends on modeling task (regression/classification)

- Memory/storage capacity: dimension of $\mathbf{h} \in \mathbb{R}^{d_h}$

Parameters are shared across time
same weights $W_{(\cdot)}$ apply for every time step

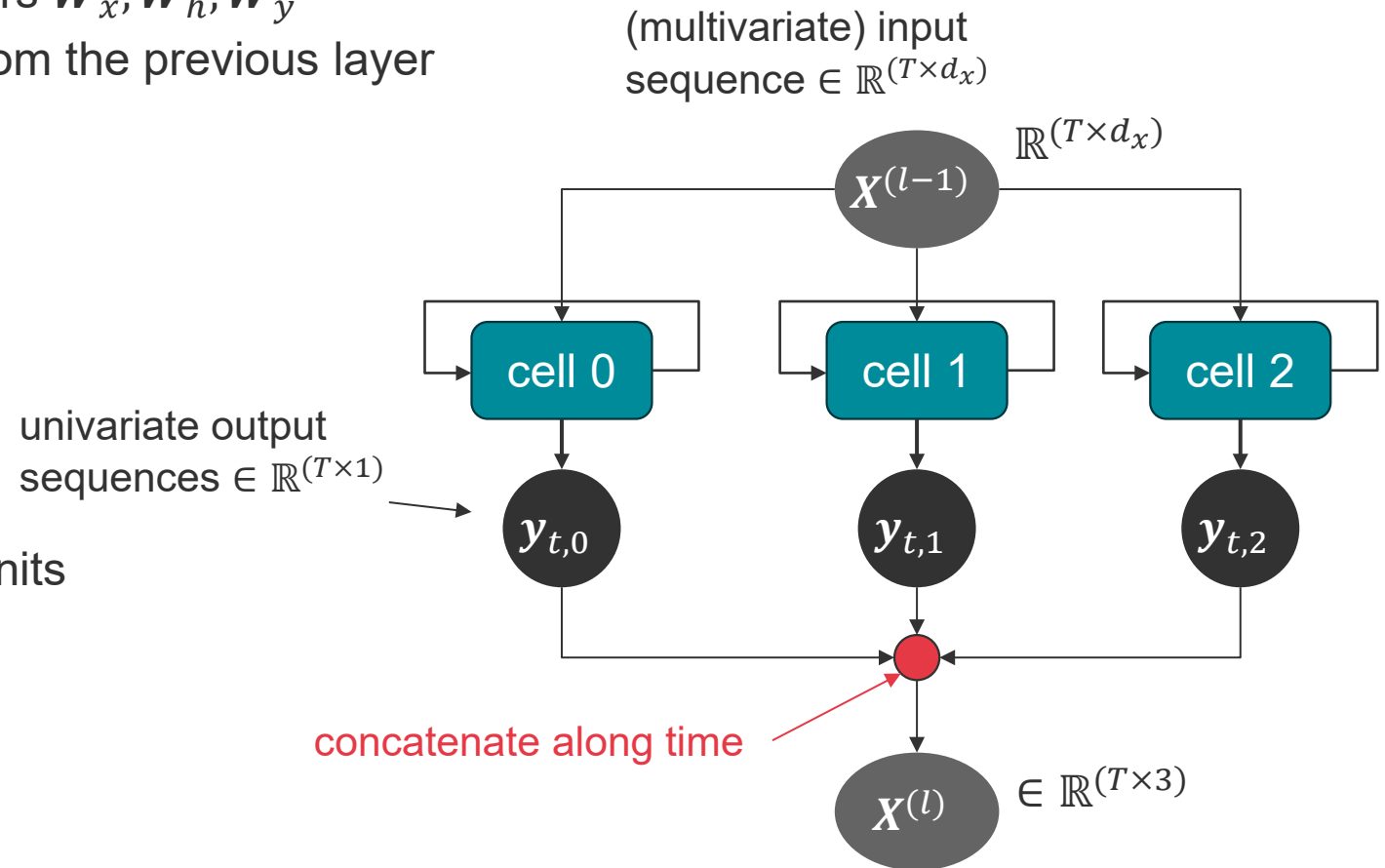
→ Unrolling creates depth in time, but not new parameters.

Recurrent Layers and Networks



- A recurrent neural layer has multiple recurrent units. Each unit:
 - has its own trainable parameters W_x, W_h, W_y
 - receives the input sequence from the previous layer
 - returns a univariate sequence

- Output channel dim = # recurrent units
- `return_sequences = True`
- `return_sequences = False`
 - returns only last element y_T

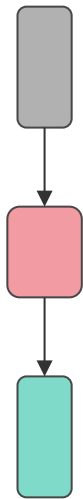


RNN: mapping scenarios



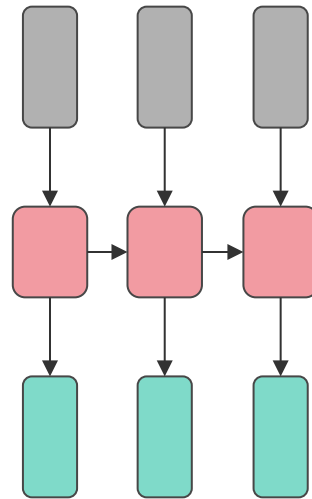
Various high-level mapping scenarios can be realized with RNNs

one to one



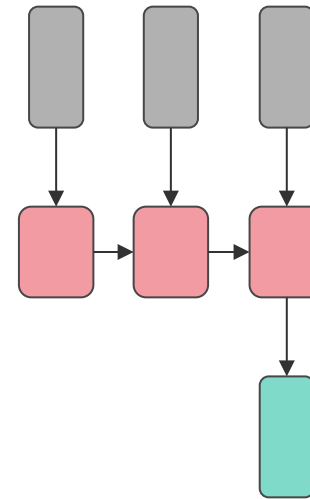
Vanilla neural network: no need to use RNN instead of feedforward NN

many to many



Language translation: multiple words mapped to multiple words

many to one



Sentiment analysis: multiple words as inputs mapped to single emotion

one to many

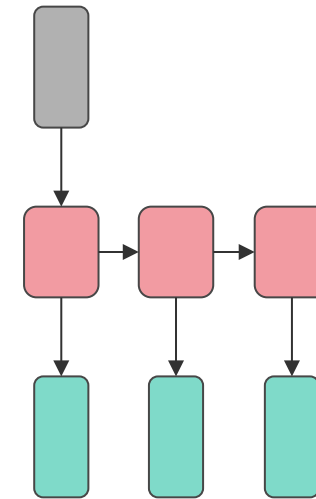


Image captioning: single input image mapped to multi-word sentence

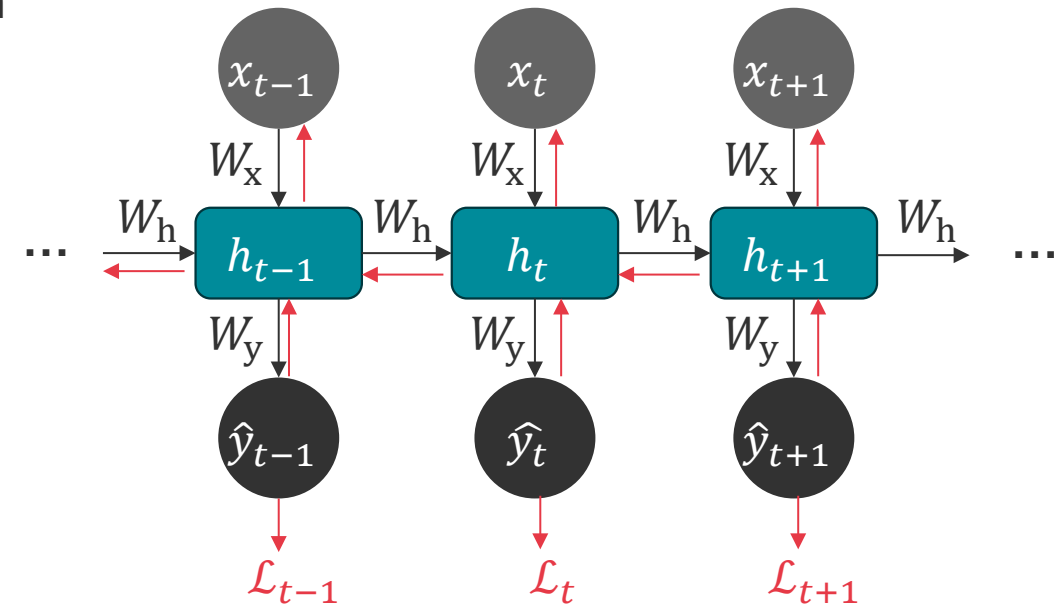


Vanishing / Exploding Gradients

Backpropagation through Time (BPTT)



- Training RNNs fully aligns with error backpropagation for feed-forward models. Additional efforts:
 - RNN cells are unrolled along time
 - Errors at each time step are computed and added up
- **Procedure:**
 1. Forward pass (as usual)
 2. Start the backward pass by calculating the gradient of the loss with respect to the output at the last time step $\frac{\partial \mathcal{L}}{\partial y_T}$
 3. Proceed backward in time, using the chain rule to calculate the gradients of the loss with respect to the parameters of the model at each time step.
 4. Weight update (as usual)



$$\mathcal{L} = \sum \mathcal{L}_{t_i}$$

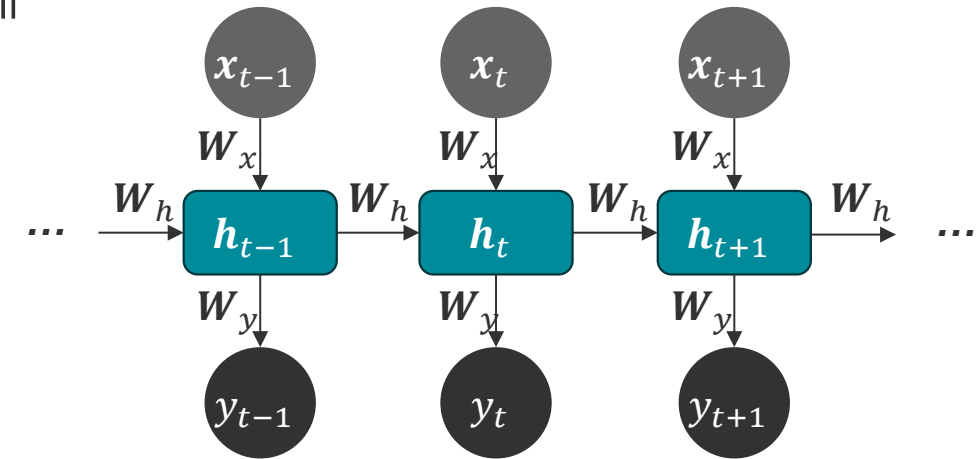
Gradients



- Consider the loss for a many-to-many task $\mathcal{L} = \sum_t \|\hat{\mathbf{y}}_t - \mathbf{y}_t\|$

$$\mathbf{h}_t = g_1(\mathbf{W}_x \mathbf{x}_t + \mathbf{b} + \mathbf{W}_h \mathbf{h}_{t-1}) \rightarrow y_t = g_2(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

- Last output element y_T depends on all inputs $x_{t=1,\dots,T}$ and all hidden states $h_{0,\dots,T}$
- Backpropagation for $\mathbf{W}_0$ requires to step through the complete sequence



- Simplified scalar model at time point t within that sequence:

$$h_t = \alpha h_{t-1} + \beta x_t \Rightarrow \frac{\partial h_t}{\partial h_{t-1}} = \alpha$$

$$h_t = \alpha h_{t-1} + \beta x_t = \alpha(\alpha h_{t-2} + \beta x_{t-1}) + \beta x_t \Rightarrow \frac{\partial h_t}{\partial h_{t-2}} = \alpha^2 \quad \dots \rightarrow \frac{\partial h_t}{\partial h_{t-K}} = \alpha^K$$

some numbers... $0.9^{10} = 0.35$, $0.9^{50} = 0.005$, $1.1^{50} = 117$. → vanishing / exploding!

Vanishing/Exploding Gradient Problem



- Measures commonly employed for helping vanishing/exploding gradients (also for LSTM)
- **Exploding gradient clipping:** To mitigate exploding gradient problems in RNNs, gradients are scaled if they exceed a certain threshold
- **Usage of tanh activation** to counteract vanishing gradients
- **Truncated backpropagation through time:** Instead of backpropagating through the entire sequence, the gradient is truncated after a certain number of time steps. Helps both vanishing and exploding gradients. Limits the long-term memory capabilities.
- And many more...
 - Weight initialization (He initialization)
 - Batch normalization
 - Skip (residual) connections
 - Gradient normalization
 - Regularization and adaptive learning rates (adam, RMSprop)



LSTM

Overcoming vanishing or exploding gradients

The design response



- We want an **additive** state path whose coefficient can be learned separately for each state component and time step.
- In classic RNNs, the state h must
 - store everything useful from the past
 - Provide a representation needed for the current output
- Every time step t completely rewrites the state, so gradients must flow backwards through time
- LSTM separates these responsibilities



ITU/R.Farrell -
<https://www.flickr.com/photos/itupictures/34343385563>, CC BY 2.0

Jürgen Schmidhuber (1963)

Co-father of LSTM networks

Scientific director
IDSIA Swiss AI Research Lab

Director
KAUST (King Abdullah University of
Science and Technology)

Sepp Hochreiter (1967)

Co-father of LSTM networks

Actively pushing for next-generation AI

Professor Johannes Kepler University Linz
Founder of NXAI



© JKU



Long short-term memory units (LSTMs)



- Proposed in **1997** by S. Hochreiter und J. Schmidhuber „Long short-term memory.” in Neural Computation 9(8):1735–1780
- **Idea:** Improve the cell state update $h_t = f(x_t, h_{t-1})$ for **selectively learning and storing information over long sequences**: Keep hold of relevant context (memorable information), but also forget irrelevant information
 - **Cell state:** memorizes long term information
 - **Forget gate:** which state information can be forgotten (i.e. is contextually irrelevant)
 - **Input gate:** what new information should be updating the hidden cell state
 - **Output gate:** which of the state information should be part of the output
- **LSTMs help to build more memory** (allows for larger temporal gaps between relevant input and desired output sequence position) to capture long-term dependencies



LSTM: 2 state vectors



- At time t , a LSTM cell receives $\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{C}_{t-1}$ and produces $\mathbf{h}_t, \mathbf{C}_t$
- Two states with different responsibilities
 - $\mathbf{C}_t$: internal cell state (main **memory path** through time) → „stores“ *
 - $\mathbf{h}_t$: exposed hidden state for computing the output → „communicates“ *
- Define $\mathbf{q}_t = [\mathbf{h}_{t-1}, \mathbf{x}_t]^\top \in \mathbb{R}^{d_h+d_x}$ as a central point for information access of gates
- **Gates** have the general form $\mathbf{g}_t = \sigma(\mathbf{W}_g \mathbf{q}_t + \mathbf{b}_g) \in (0,1)^{d_h}$, sigmoid σ acts like a soft switch
→ based on $\mathbf{W}_g$ one can retain/overwrite/update one or both states $\mathbf{h}_t, \mathbf{C}_t$

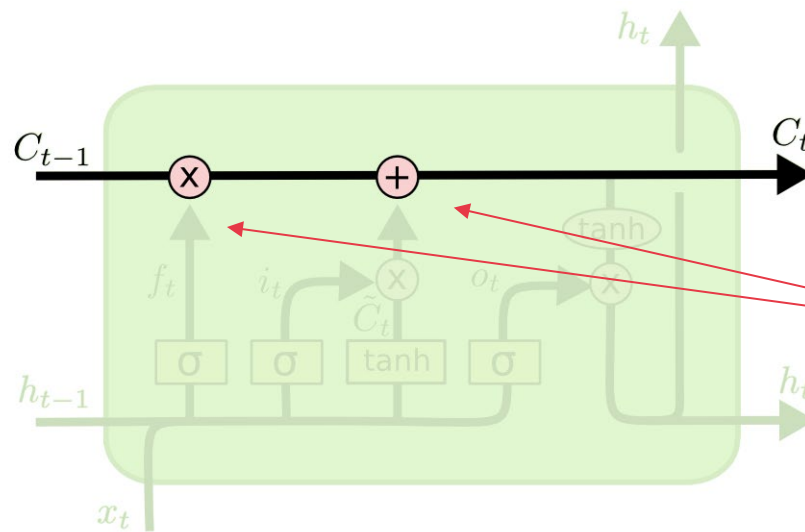
→ Forget gate, input gate, output gate

*no perfect separation (both involved in future computations, but useful for pedagogical abstraction)

Long short-term memory units (LSTMs)



- The cell state: flow of information that may get updated



The cell state C cannot be changed easily by the LSTM unit – gates control the information flow

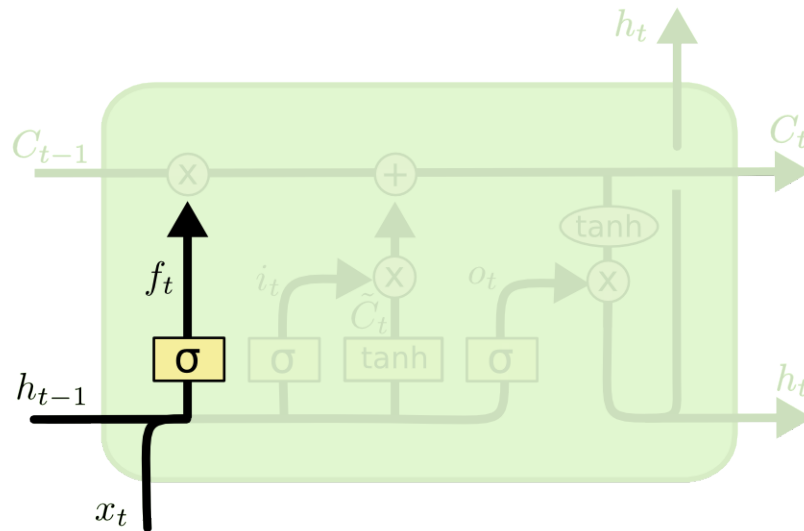
Gates (sigmoid activated neural net layer)

Image from Colah's blog
<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Long short-term memory units (LSTMs)



- Forget gate f_t looking at the previous hidden state h_{t-1} and the current input x_t



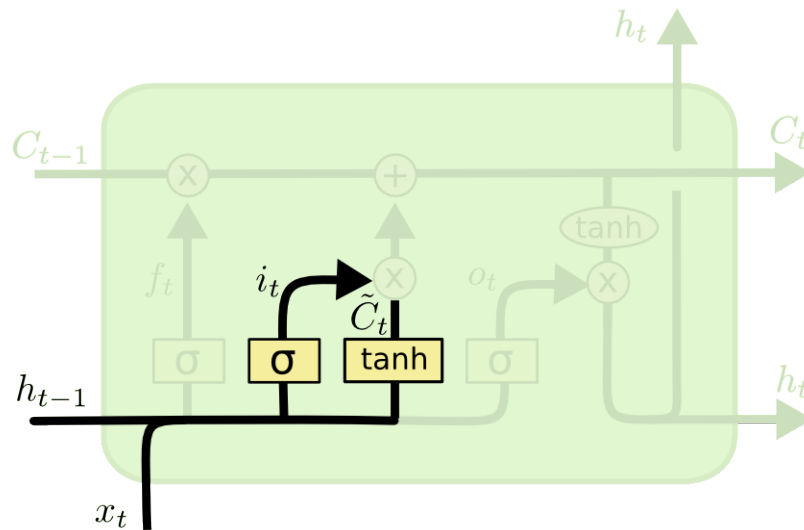
$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Image from Colah's blog
<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Long short-term memory units (LSTMs)



- Input gate i_t looking at the previous hidden state h_{t-1} and the current input x_t to update the cell state



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$\tilde{C}_t$: candidate cell state (new values to update the cell state)

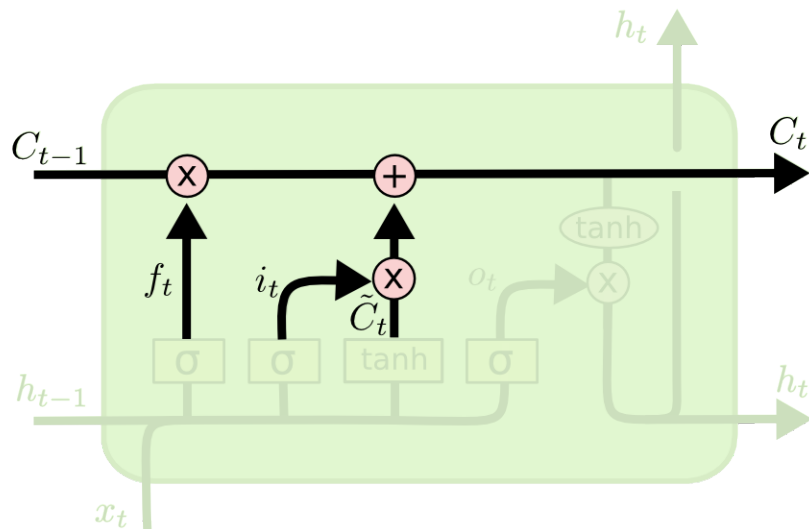
Image from Colah's blog
<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Long short-term memory units (LSTMs)



Cyber-Physical Systems
in Mechanical Engineering TU Berlin

- Perform the actual cell state update (forgetting / adding information)



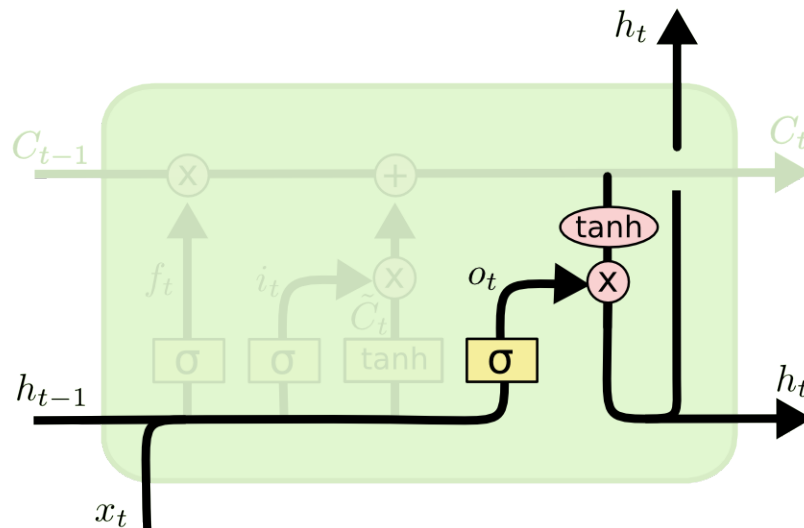
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Image from Colah's blog
<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Long short-term memory units (LSTMs)



- Decide on the next hidden state h_t from the cell output o_t



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

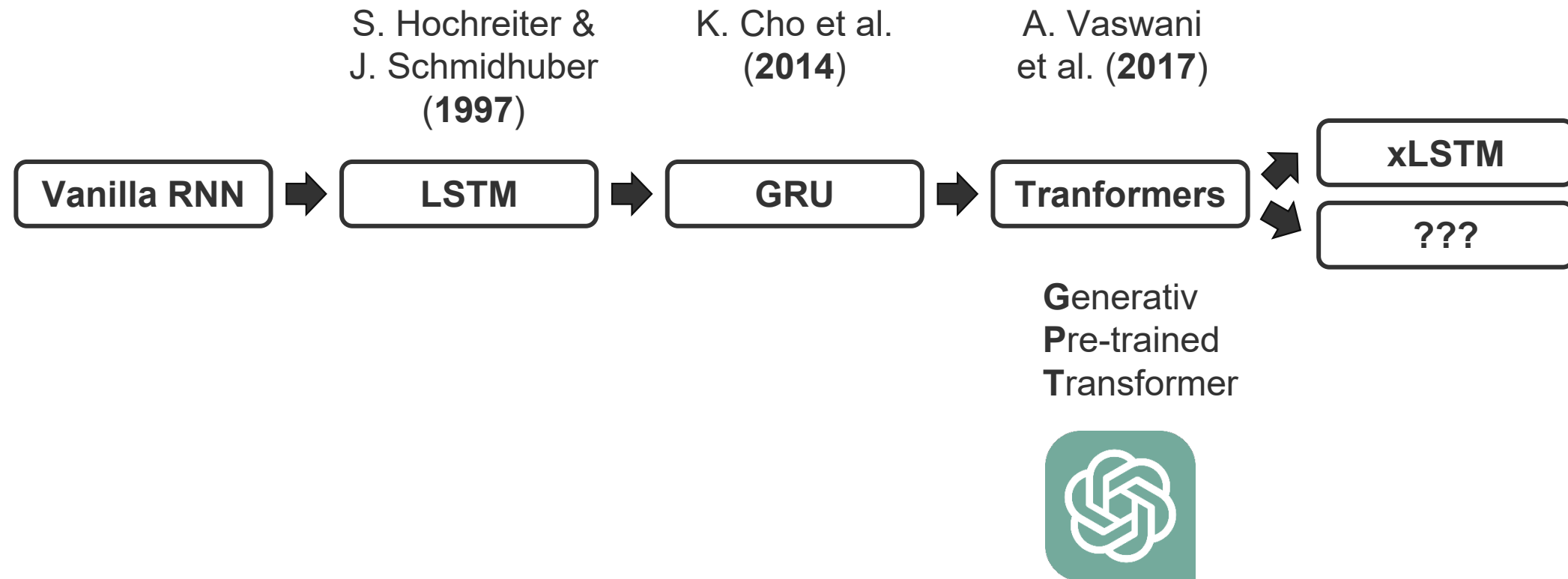
Image from Colah's blog
<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>



History and Future of RNNs



- A high-level overview on neural network architectures for sequential data





xLSTM: cutting edge?

„I am almost exploding“

Industrial AI Podcast ([YouTube](#), Nov. 2023)

„Wir werden das blöde GPT einfach wegstossen“

(„We’re going to smash this dump GPT“ ([WELT](#), Oct. 2023)

Publication of the xLSTM architecture

(<https://arxiv.org/abs/2405.04517> , May 2024)





GRU



Gated Recurrent Units (GRU)



- Proposed in **2014** by K. Cho; B. van Merriënboer; D. Z. Bahdanau; F. Bougares; H. Schwenk; Y. Bengio "[Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation](#)". Association for Computational Linguistics.
- Motivation: **Combine forget and input gate** of LSTM into a single update gate. Do not keep a separate cell state, but contain **all information in the hidden state**
- **Advantages:**
 - Simpler architecture (faster training)
- **Disadvantage:**
 - Not as capable as LSTMs for very long-term memory

Gated Recurrent Units (GRU)



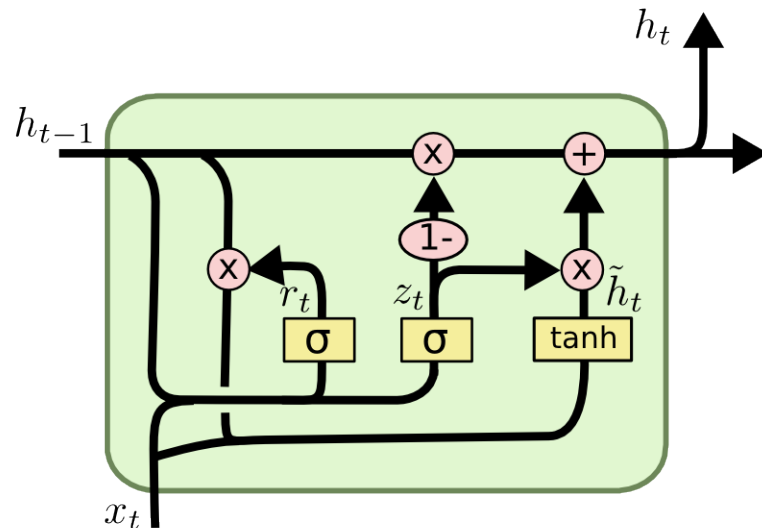
Main components of GRU:

- **Hidden state** h_t : usual hidden state capturing information from previous time steps
- **Candidate hidden state** h'_t : represents the new information that might be added to the hidden state, computed by considering the current input and the reset gate. tanh activation
- **Update gate** z_t : determines the extent to which the hidden state from the previous time step should be mixed with the candidate new hidden state. Sigmoid activation.
- **Reset gate** r_t : controls the extent to which the previous hidden state h_{t-1} should be forgotten when computing the candidate new hidden state h'_t . Sigmoid activation.
- **Weighted combination**: new hidden state h_t is computed by combining the previous hidden state h_{t-1} and the candidate hidden state h'_t based on the update gate z_t .
- **Hidden state update**: $h_t = (1 - z_t)h_{t-1} + z_th'_t$

Gated Recurrent Units (GRU)



- Combination of the cell state and the hidden state
- Combination of the forget and input gate



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Image from Colah's blog
<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>



Questions?